University of Essex

CE205
Databases and Information Retrieval

Vector Space Model and Evaluation of IR System

Dr. Zulfiqar Ali

Lecturer

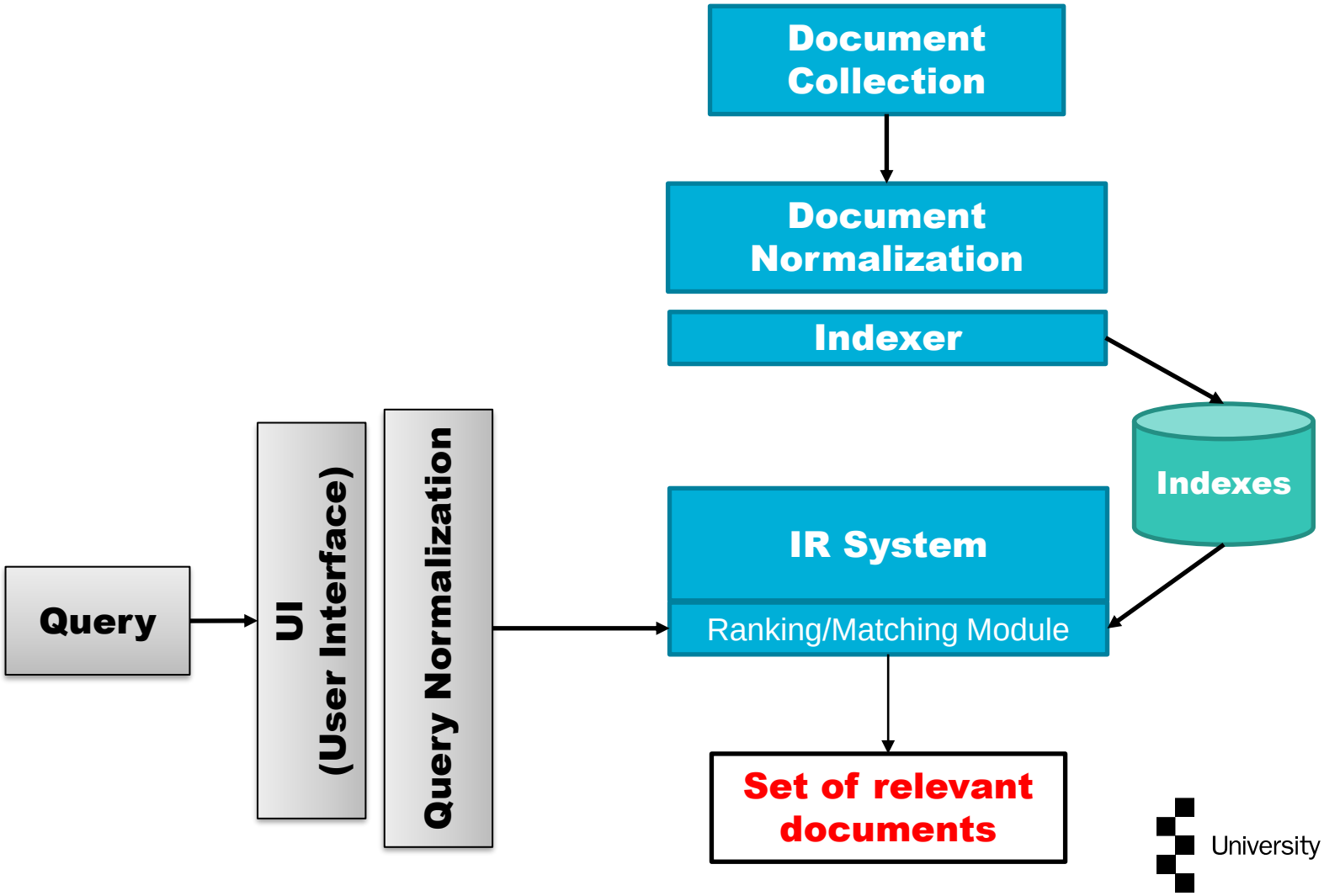
School of Computer Science and Electronic Engineering

- *[Note: Some slides in this set were adapted from an IR course taught by Rada Mihalcea at the University of Michigan and Ray Mooney at UT Austin (who in turn adapted them from Joydeep Ghosh), and from an IR course taught by Chris Manning at Stanford)]*

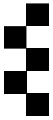
Learning Objectives

- Scoring of a document for a given query
- Term Weighting
  - What is Term Frequency (TF) weight?
  - What is Inverse Document Frequency (IDF) weight?
  - What is TF-IDF weights ?
  - How to compute these weights?
- What is vector space model in IR system?
- Why evaluation of an IR system is important using various performance measures *(if time permits)*

Where were we → Where we are now



Scoring



University of Essex

Ranked Retrieval Models

- Rather than a set of documents satisfying a query expression, in ranked retrieval, the system returns an ordering over the (top) documents

- Free text queries: Rather than a query language of operators and expressions, the user's query is just **one or more words in a human language**

Scoring as the Basis of Ranked Retrieval

- How can we rank (order) the documents in the collection with respect to a query?
- Assign a score – say in [0, 1] – to each document

- This score measures how well document and query "match".

Query-document Matching Scores

- We need a way of assigning a score to a query/document pair
- Let's start with a one-term query
- If the query term does not occur in the document: score should be 0
- The more frequent the query term in the document, the higher the score (should be)

Jaccard coefficient

- Jaccard: A commonly used measure of overlap of two sets A and B
- Jaccard(A,B) = |A ∩ B| / |A ∪ B| *where |A ∩ B| means number of elements in A ∩ B.*
- Jaccard(A,A) = 1
- Jaccard(A,B) = 0 if A ∩ B = 0
- A and B don't have to be the same size.
- Always assigns a number between 0 and 1.

Exercise: Jaccard Coefficient

- What is the query-document match score that the Jaccard coefficient computes for each of the two documents below?
- **Query**: march of dimes
- **Document 1**: caesar died in march
- **Document 2**: the long march

Jaccard(Q,D1)=?

Jaccard(Q,D2)=?

Exercise: Jaccard Coefficient

Solution:

- **Query**: march of dimes
- **Document 1**: caesar died in march
- Q∩D1={march}
- Q∪D1={march, of ,dimes, Caesar, died, in}

$$\text{Jaccard}(Q,D1) = \frac{|Q \cap D1|}{|Q \cup D1|}$$

$$= \frac{1}{6} = 0.167$$

Similarly, Jaccard(Q,D2)= 0.2

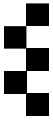
University of Essex

Issues with Jaccard for Scoring

- It does not consider term frequency (how many times a term occurs in a document)

- Rare terms in a collection are more informative than frequent terms. ***Jaccard coefficient does not consider this information.***

tf-idf Weighting



University of Essex

Term Frequency tf

- The term frequency $tf_{t,d}$ of term *t* in document *d* is defined as the number of times that term *t* occurs in document *d*.
- We want to use tf when computing query-document match scores. But how?
- Raw term frequency is not what we want:
  - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
  - But not 10 times more relevant.
- Relevance does not increase proportionally with term frequency.

Document Frequency (1)

- Rare terms are more informative than frequent terms
  - Recall stop words
- Consider a term in the query that is rare in the collection (e.g., *arachnocentric*)
- A document containing this term is very likely to be relevant to the query *arachnocentric*
  - **We want a high weight for rare terms like *arachnocentric*.**

Document Frequency (2)

- **Frequent terms are less informative than rare terms**
- Consider a query term that is frequent in the collection (e.g., high, increase, line)
  - A document containing such a term is more likely to be relevant than a document that doesn't.
- But it's not a sure indicator of relevance.
  - **For frequent terms, we want high positive weights for words like high, increase, and line but lower weights than for rare terms.**
- **We will use document frequency (df) to capture this.**

Inverse Document Frequency (IDF)

- **df_t** is the document frequency of *t*: the number of documents that contain *t*
  - df_t is an inverse measure of the informativeness of *t*
  - $df_t \leq N$ *(number of documents in collection)*
- **idf** (inverse document frequency) is the *Reciprocal* of how often Term appears in entire doc collection: 1/df

idf (non-normalised) example, suppose N = 1 million

term	df_t	idf_t
calpurnia	1	1
animal	100	0.01
sunday	1,000	0.001
fly	10,000	0.0001
under	100,000	0.00001
the	1,000,000	0.000001

$$\text{idf}_t = \frac{1}{df_t}$$

There is one idf value for each term *t* in a collection.

tf-idf Weighting (1)

- The higher the frequency of a term in a doc, the better the doc matches our query term.
- The higher the frequency of the term in docs, generally, the worse it is as a search term.
    - IDF becomes worse for us as it becomes more frequent.
- We want the best 'combination' of these factors.
- **Multiply tf by idf** to get an overall weight for term.

tf-idf Weighting (2)

The **tf-idf weighting scheme** assigns to term *t* a weight TF-IDF in document *d* given by

$$\text{tf-idf}_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

- **highest when term *t* occurs many times within a small number of documents;**
- **lower when the term occurs fewer times in a document, or occurs in many documents;**
- **lowest when the term occurs in virtually all documents.**

Note: the "-" in tf-idf is a hyphen, not a minus sign!

Alternative names: tf.idf, tf x idf

Binary → Count → Weight matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Each document is now represented by a real-valued vector of tf-idf weights $\in R^{|V|}$

tf-idf Score for a Document given a Query

- The score of a document *d* is the sum, over all query terms, of the number of times each of the query terms occurs in *d*.

$$\text{Score}(q,d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

Log-frequency weighting

- The log frequency weight of term *t* in *d* is

$$w_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{tf}_{t,d}, & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- 0 → 0, 1 → 1, 2 → 1.3, 10 → 2, 1000 → 4, etc.
- *The weight is 0 if none of the query terms is present in the document.*

idf example, documents in collection (N) are 1 million

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

$$\text{idf}_t = \log_{10}\left(\frac{N}{df_t}\right) = \log_{10}\left(\frac{1000000}{df_t}\right)$$

There is one idf value for each term *t* in a collection.

N represents number of documents in collection

idf example, documents in collection (N) are 18

Term	Document Frequency	Collection Frequency	IDF
the	18	133,572	0.0
people	17	2,781	0.0248
thee	11	4,807	0.2139
truths	7	14	0.4101
fixing	5	14	0.5563
manifested	2	14	0.9542
vigil	1	13	1.2553

idf example, N = 1 million
(another way of computing)

term	df_t	idf_t
calpurnia	1	
animal	100	
sunday	1,000	
fly	10,000	
under	100,000	
the	1,000,000	

$$\text{idf}_t = \log_2 \left(\frac{N}{n_t + 1} \right)$$

n_t represents total occurrences of term *t* in collection

N represents number of documents in collection

University of Essex

tf-idf weight
(another way of computing)

tf-idf weight is product of **tf weight** and **idf weight** and can also be calculated as

tf-idf weight

$$w_{t,d} = (1 + \log \mathrm{tf}_{t,d}) \cdot \log \frac{N}{\mathrm{df}_t}$$

Computing tf-idf: An Example

- Given a document (D) containing terms with given frequencies:

  A(3), B(2), C(1)

- Assume collection contains 10,000 documents and document frequencies of these terms are:

  A(50), B(1300), C(250)

Determine tf-idf weight for the terms A, B and C using the following formula

$$\text{weight}_{td} = tf_{td} \times \log_{10}\left(\frac{N}{df_t}\right)$$